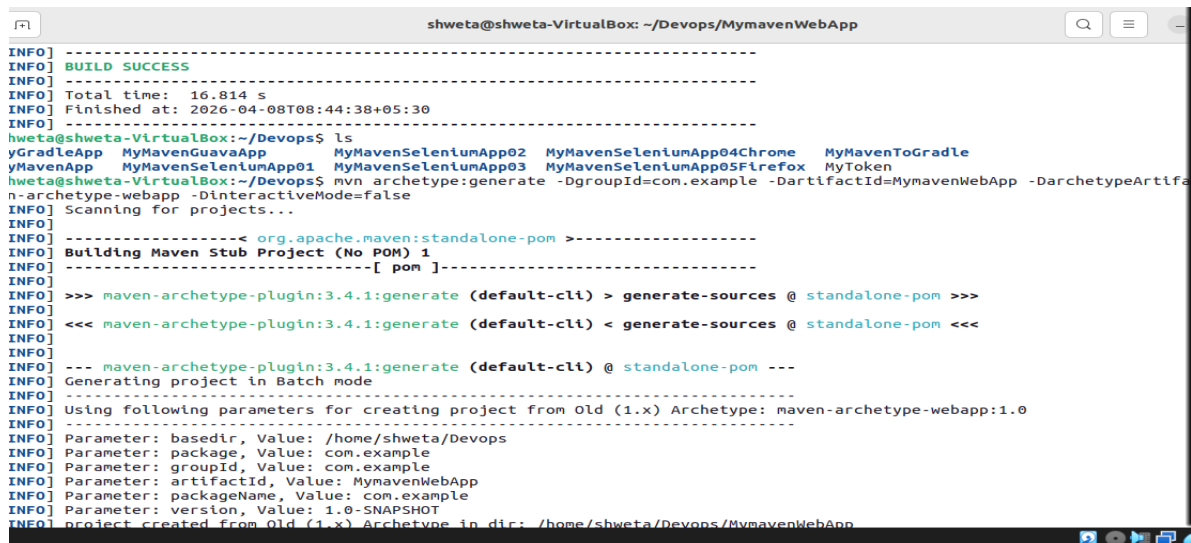


Program 10: Creating Maven Web App:

Step 1: This below Maven command creates a new web application project named **MyMavenWebApp** using the **maven-archetype-webapp** template. It sets the group ID to com.example and runs in non-interactive mode, generating all project files automatically.

```
mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenWebApp -DarchetypeArtifactId=maven-archetype-webapp -DinteractiveMode=false
```



```
shweta@shweta-VirtualBox: ~/Devops/MyMavenWebApp
INFO] -----
INFO] BUILD SUCCESS
INFO] -----
INFO] Total time: 16.814 s
INFO] Finished at: 2026-04-08T08:44:38+05:30
INFO] -----
shweta@shweta-VirtualBox:~/Devops$ ls
yGradleApp  MyMavenGuavaApp  MyMavenSeleniumApp02  MyMavenSeleniumApp04Chrome  MyMavenToGradle
yMavenApp  MyMavenSeleniumApp01  MyMavenSeleniumApp03  MyMavenSeleniumApp05Firefox  MyToken
shweta@shweta-VirtualBox:~/Devops$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenWebApp -DarchetypeArtifactId=maven-archetype-webapp -DinteractiveMode=false
INFO] Scanning for projects...
INFO]
INFO] -----< org.apache.maven:standalone-pom >-----
INFO] Building Maven Stub Project (No POM) 1
INFO] -----[ pom ]-----
INFO]
INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
INFO]
INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
INFO] Generating project in Batch mode
INFO]
INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-webapp:1.0
INFO] -----
INFO] Parameter: basedir, Value: /home/shweta/Devops
INFO] Parameter: package, Value: com.example
INFO] Parameter: groupId, Value: com.example
INFO] Parameter: artifactId, Value: MyMavenWebApp
INFO] Parameter: packageName, Value: com.example
INFO] Parameter: version, Value: 1.0-SNAPSHOT
INFO] project created from Old (1.x) Archetype in dir: /home/shweta/Devops/MyMavenWebApp
```

Step 2: The **tree** command output shows the hierarchical layout of the Maven web application project **MyMavenWebApp**, highlighting its main directories, key files like pom.xml and web.xml, and the placement of web resources (index.jsp) under src/main/webapp. It reflects a standard Maven webapp setup with 6 directories and 3 files.

```

shweta@shweta-VirtualBox: ~/Devops/MyMavenWebApp
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO] -----
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-webapp:1.0
[INFO] -----
[INFO] Parameter: basedir, Value: /home/shweta/Devops
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenWebApp
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/shweta/Devops/MyMavenWebApp
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.353 s
[INFO] Finished at: 2020-04-08T08:49:00+05:30
[INFO] -----
shweta@shweta-VirtualBox:~/Devops$ ls
MyGradleApp  MyMavenGuavaApp  MyMavenSeleniumApp02  MyMavenSeleniumApp04Chrome  MyMavenToGradle  MyToken
MyMavenApp   MyMavenSeleniumApp01  MyMavenSeleniumApp03  MyMavenSeleniumApp05Firefox  MyMavenWebApp
shweta@shweta-VirtualBox:~/Devops$ cd MyMavenWebApp
shweta@shweta-VirtualBox:~/Devops/MyMavenWebApp$ ls
pom.xml  src
shweta@shweta-VirtualBox:~/Devops/MyMavenWebApp$ tree
.
├── pom.xml
└── src
    ├── main
    │   ├── resources
    │   │   ├── webapp
    │   │   │   ├── index.jsp
    │   │   │   └── WEB-INF
    │   │   │       └── web.xml
    └── test
5 directories, 3 files

```

Step 3: gedit pom.xml

This POM.xml configures the Maven web application **MyMavenWebApp**.

- **JUnit 3.8.1:** A testing tool used to check that the code works correctly. It runs small tests on your program automatically.
- **Servlet API 4.0.1:** Provides the classes needed to create web features like handling user requests, forms, and web pages. The server (like Tomcat) gives it at runtime.
- **maven-war-plugin 3.4.0:** Helps build the project into a **WAR file**, which is the package needed to deploy the web application on a web server.

This file also contains project information, version, and build settings needed to manage and deploy the web app easily.

```

<project
xmlns="http://maven.apache.org/POM/4.0.0
"
```

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"

xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.ap
ache.org/maven-v4_0_0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId>

<artifactId>MymavenWebApp</artifactId>

<packaging>war</packaging>

<version>1.0-SNAPSHOT</version>

<name>MymavenWebApp Maven Webapp</name>

<url>http://maven.apache.org</url>

<dependencies>

<dependency>

<groupId>junit</groupId>
<artifactId>junit</artifactId>

<version>3.8.1</version>

<scope>test</scope>

</dependency>

<!-- Servlet API for Java EE -->

<dependency>
```

```
<groupId>javax.servlet</groupId>

<artifactId>javax.servlet-api</artifactId>

<version>4.0.1</version>

<scope>provided</scope>

</dependency>

</dependencies>

<build>

<plugins> <plugin>

  <groupId>org.apache.maven.plugins</groupId>

  <artifactId>maven-war-plugin</artifactId>

  <version>3.4.0</version> <!-- Update to latest stable version -->

</plugin>

</plugins>

  <finalName>MymavenWebApp01</finalName>

</build>

</project>
```

Step 4: mvn clean install

The mvn clean install command cleans previous builds, compiles the code, runs tests, packages the application, and installs it into the local Maven repository, making it ready for deployment.

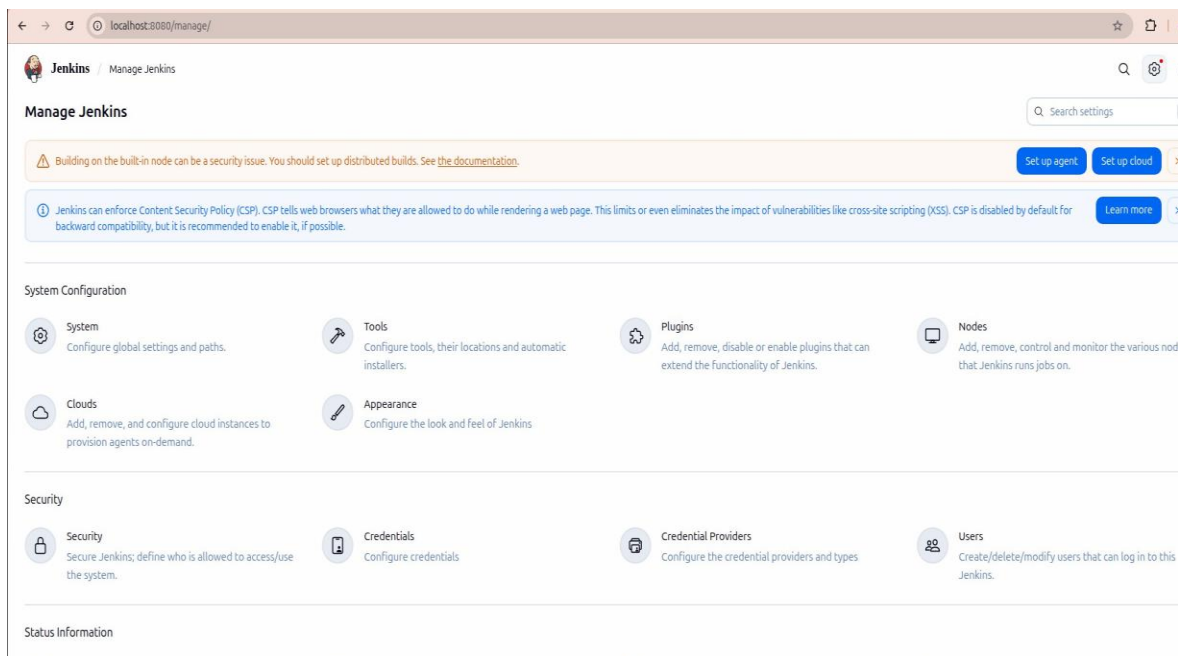
```

shweta@shweta-VirtualBox: ~/Devops/MymavenWebApp
[INFO] Assembling webapp [MymavenWebApp] in [/home/shweta/Devops/MymavenWebApp/target/MymavenWebApp]
[INFO] Processing war project
[INFO] Copying webapp resources [/home/shweta/Devops/MymavenWebApp/src/main/webapp]
[INFO] Building war: /home/shweta/Devops/MymavenWebApp/target/MymavenWebApp.war
[INFO] --- maven-install-plugin:2.4:install (default-install) @ MymavenWebApp ---
[INFO] Installing /home/shweta/Devops/MymavenWebApp/target/MymavenWebApp.war to /home/shweta/.m2/repository/com/example/MymavenWebApp/1.0-SNAPSHOT/MymavenWebApp-1.0-SNAPSHOT.war
[INFO] Installing /home/shweta/Devops/MymavenWebApp/pon.xml to /home/shweta/.m2/repository/com/example/MymavenWebApp/1.0-SNAPSHOT/MymavenWebApp-1.0-SNAPSHOT.pom
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.252 s
[INFO] Finished at: 2026-04-08T09:02:14+05:30
[INFO] -----

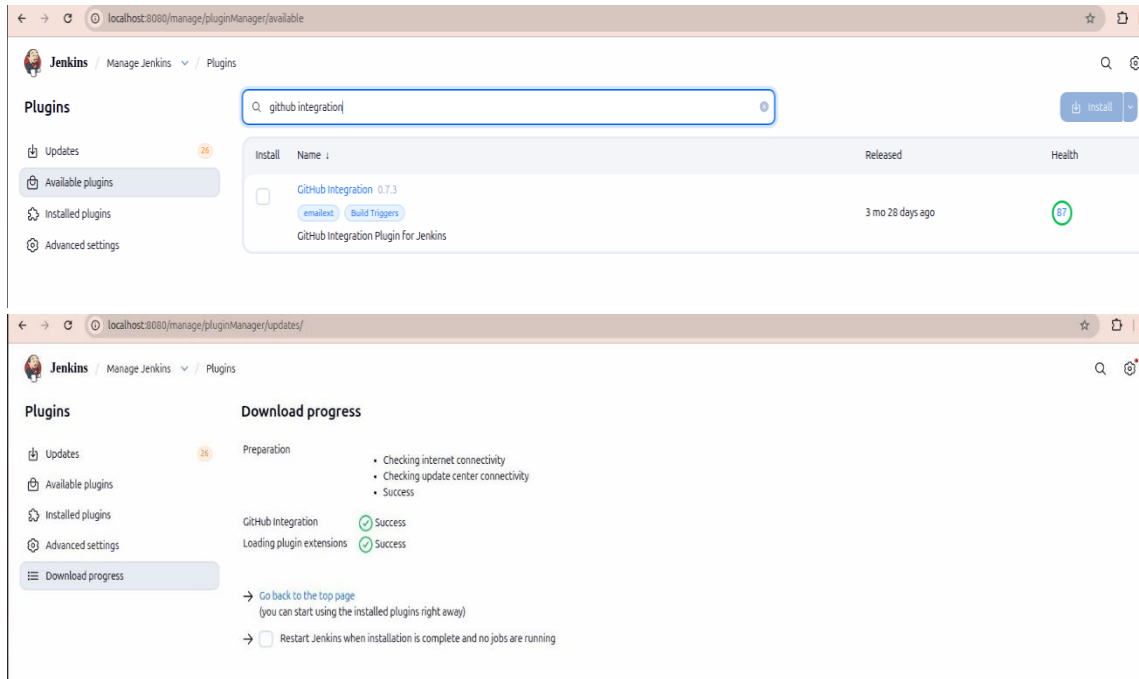
```

Step 5: To set up essential Jenkins plugins, log in to Jenkins and navigate to **Dashboard → Manage Jenkins → Plugins → Available**. Search for and install the following plugins:

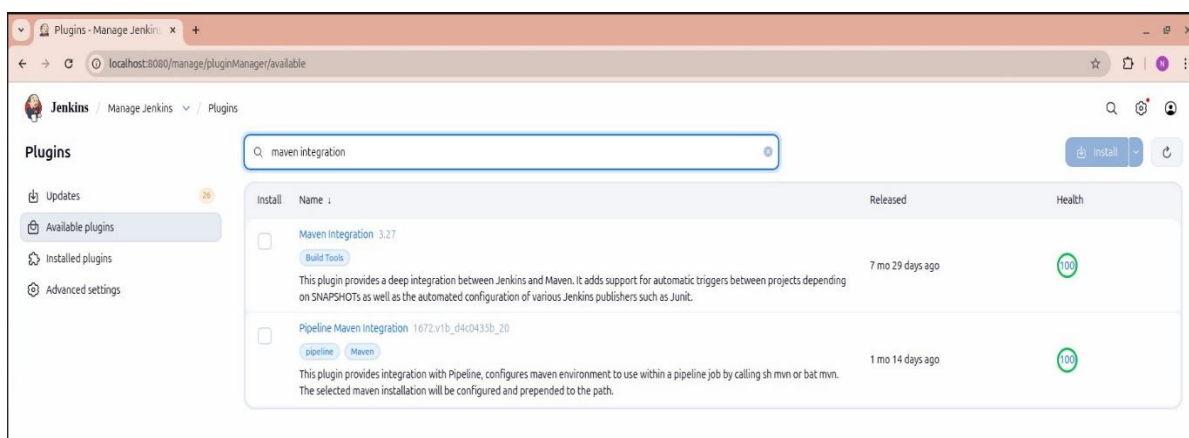
1. **Maven Integration Plugin** – Enables Jenkins to build Maven projects.
2. **Gradle Integration Plugin** – Allows building projects using Gradle.
3. **Pipeline: Stage View Plugin** – Provides a visual stage view for Jenkins pipelines.
4. **GitHub Integration Plugin** – Connects Jenkins with GitHub for source code management and triggering builds.

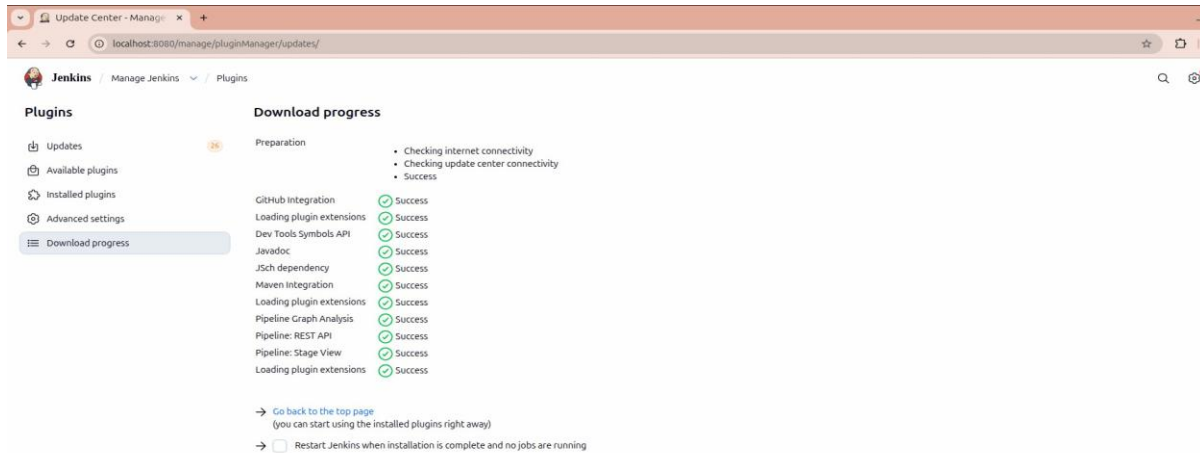


Step 6: In **Available Plugins**, search for **GitHub Integration Plugin**, check the box next to it, and then click **Install** to add it to Jenkins.

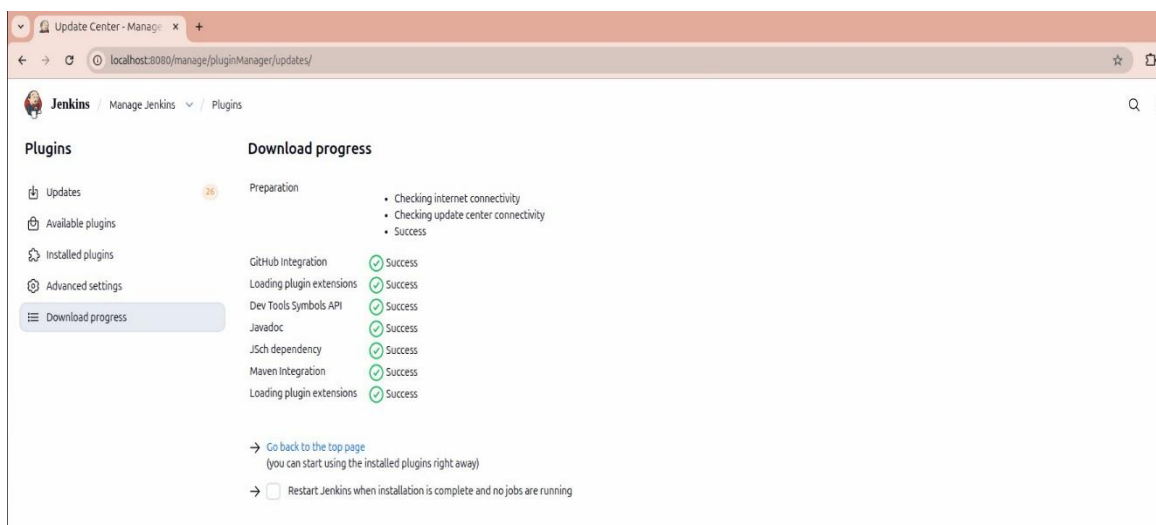
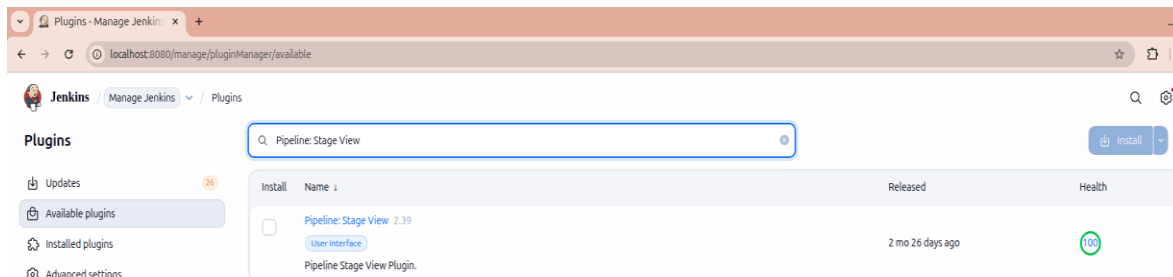


Step 7: In **Available Plugins**, search for **Maven Integration Plugin**, check the box next to it, and then click **Install** to add it to Jenkins.



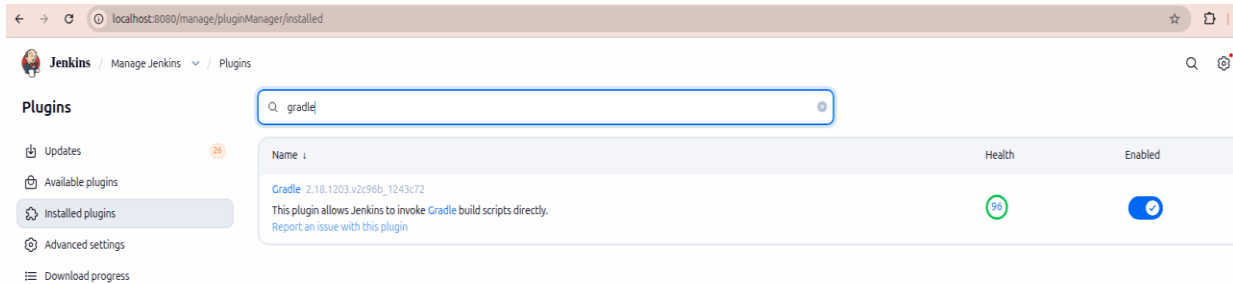


Step 7: In **Available Plugins**, search for **Pipeline: Stage View**, check the box next to it, and then click **Install** to add it to Jenkins.



Step 7: First, go to the **Installed** tab in Jenkins plugins and check if **Gradle Integration Plugin**

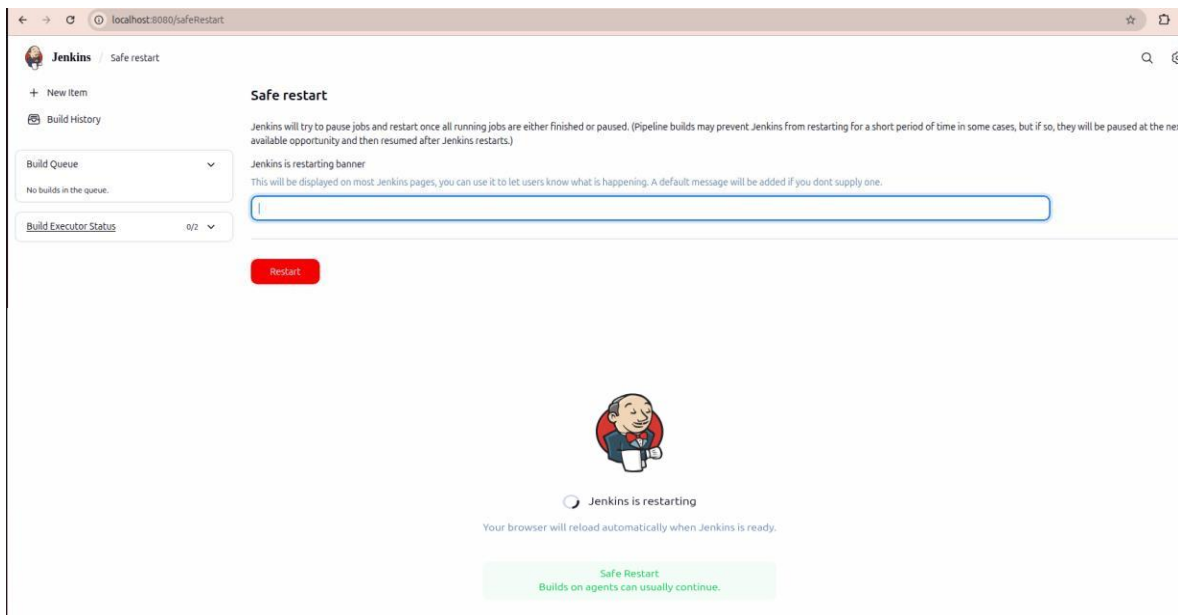
is already installed. If it is installed, no further action is needed.



Step 8: After installing the required Jenkins plugins, you need to restart Jenkins so the changes take effect. You can safely restart Jenkins by opening the link:

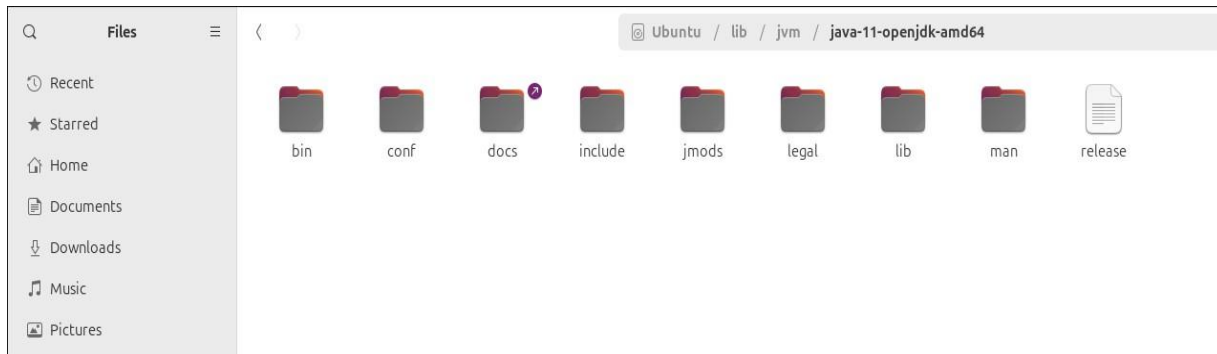
<http://localhost:8080/safeRestart>

This **safe restart** allows Jenkins to finish any running jobs before restarting, ensuring that all newly installed plugins are loaded correctly without interrupting ongoing tasks.

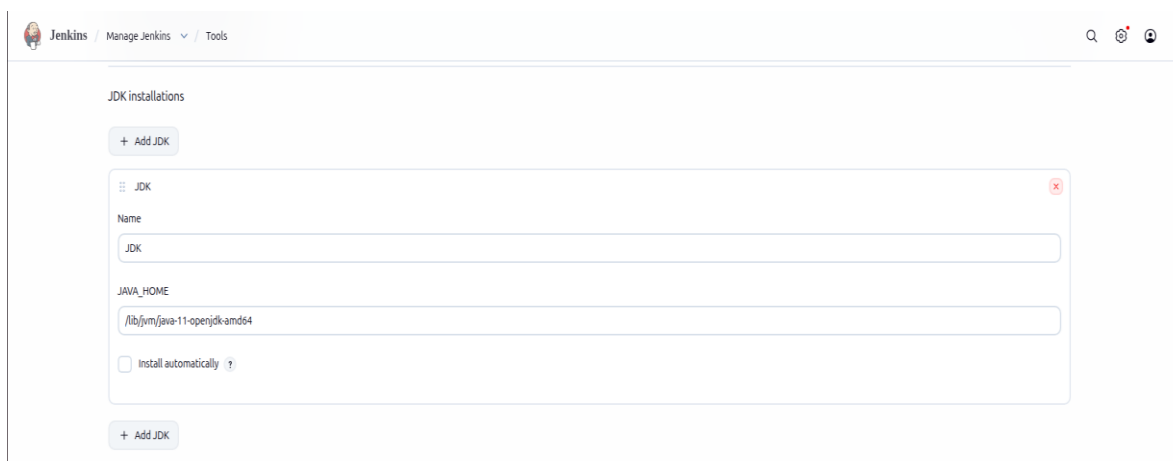


Step 9: Open Jenkins and go to **Manage Jenkins** → **Global Tool Configuration**. Leave the **Maven configuration** as default. Under **JDK**

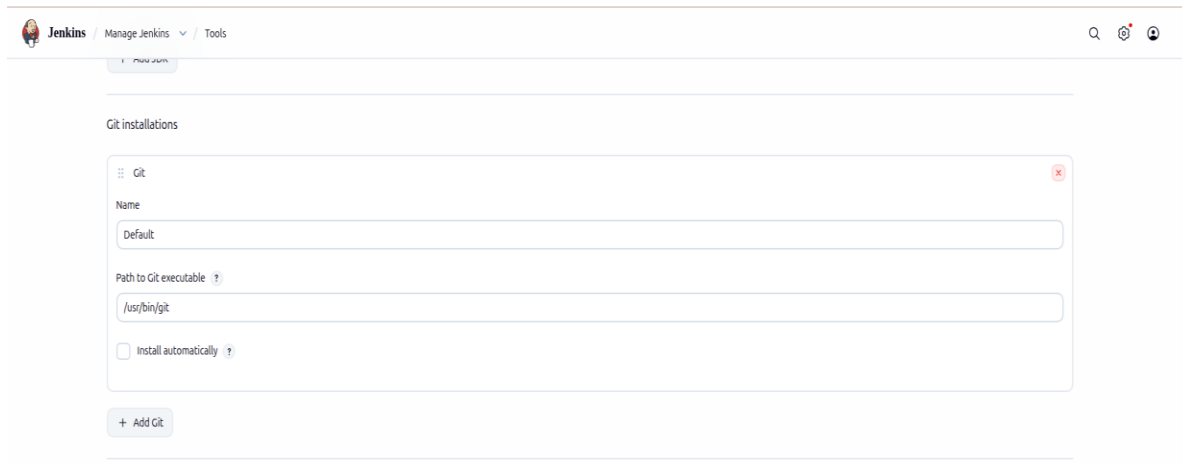
Installation, type the name as **JDK**. Then, in the terminal, run `which java`, which should show `/usr/bin/java`. This confirms Jenkins is using the correct Java path for building projects.



Copy the path `/usr/lib/jvm/java-11-openjdk-amd64` and paste it in **JAVA_HOME** under **JDK Installation** in Jenkins. Untick the **Install automatically** option to use the existing Java installation instead of letting Jenkins install it.



Step 10: Go to the terminal and type `which git`, which will show `/usr/bin/git`. Copy this path and paste it under **Git Installation** → **Path to Git executable** in Jenkins. Leave the name as default and untick **Install automatically** to use the existing Git installation.

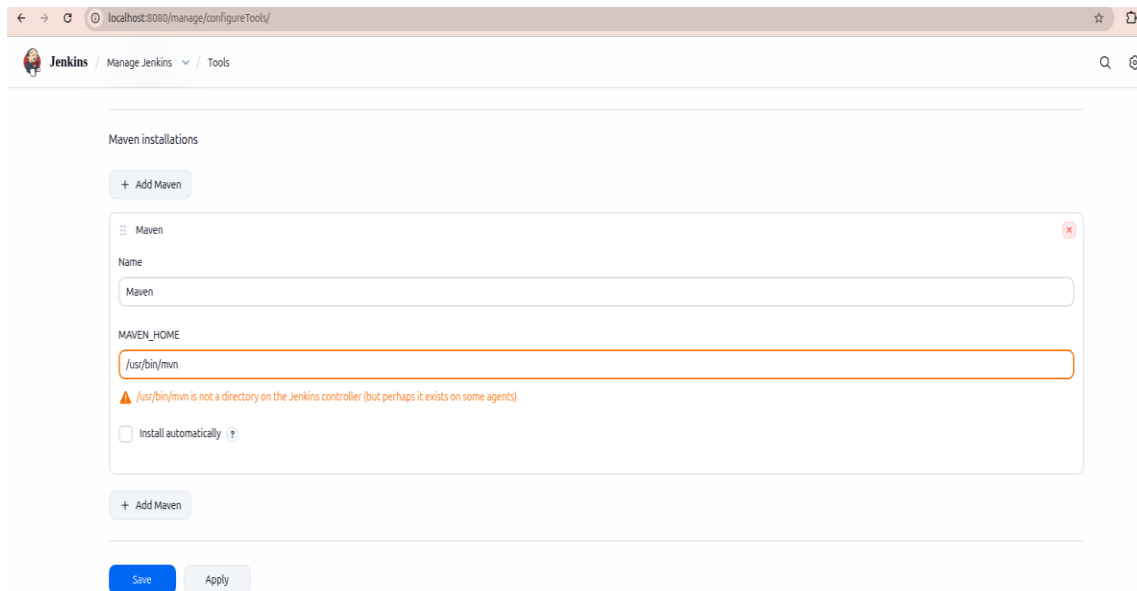


Step 11: Go to the terminal and type which gradle, which will show the Gradle path

/usr/bin/gradle. Copy this path and paste it under **Gradle Installation** → **Path to Gradle** in Jenkins. Type the name as **Gradle** and untick **Install automatically** to use the existing Gradle installation.



Step 12: Go to the terminal and type `which mvn`, which will show the Maven path `/usr/bin/mvn`. Copy this path and paste it under **Maven Installation** → **MAVEN_HOME** in Jenkins. Type the name as **Maven** and untick **Install automatically** to use the existing Maven installation.



Step 13: At last, click **Apply** to apply the changes, and then click **Save** to permanently store the configuration in Jenkins.

Installation of Apache Tomcat and Jenkins Pipeline for

MyMavenWebApp Installation of Apache Tomcat:

Step 1: Open Terminal

- **sudo -i** → Switches the user to root (administrator) mode with full system privileges.
- **sudo apt update && sudo apt upgrade -y** → Updates the package list and upgrades all installed packages automatically without asking for confirmation.

```

Hit:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:2 http://in.archive.ubuntu.com/ubuntu jammy InRelease
Hit:3 http://dl.google.com/linux/chrome/deb stable InRelease
Hit:4 http://in.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:5 http://in.archive.ubuntu.com/ubuntu jammy-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
375 packages can be upgraded. Run 'apt list --upgradable' to see them.
sudo: upgrade: command not found
root@shweta-VirtualBox:~# cd /tmp
root@shweta-VirtualBox:/tmp# wget https://archive.org/dist/tomcat/tomcat-9/v9.0.95/bin/apache-tomcat-9.0.95.tar.gz
--2026-04-12 17:43:36-- https://archive.org/dist/tomcat/tomcat-9/v9.0.95/bin/apache-tomcat-9.0.95.tar.gz
Resolving archive.org (archive.org)... 207.241.224.2
Connecting to archive.org (archive.org)[207.241.224.2]:443... connected.
HTTP request sent, awaiting response... 404 Not Found
2026-04-12 17:43:39 ERROR 404: Not Found.

root@shweta-VirtualBox:/tmp# wget https://archive.apache.org/dist/tomcat/tomcat-9/v9.0.95/bin/apache-tomcat-9.0.95.tar.gz
--2026-04-12 17:44:19-- https://archive.apache.org/dist/tomcat/tomcat-9/v9.0.95/bin/apache-tomcat-9.0.95.tar.gz
Resolving archive.apache.org (archive.apache.org)... 65.108.204.189, 2a01:4f9:1a:a084::2
Connecting to archive.apache.org (archive.apache.org)[65.108.204.189]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 12715996 (12M) [application/x-gzip]
Saving to: 'apache-tomcat-9.0.95.tar.gz'

apache-tomcat-9.0.95.tar.gz      100%[=====] 12.13M  2.71MB/s   in 4.5s

2026-04-12 17:44:25 (2.71 MB/s) - 'apache-tomcat-9.0.95.tar.gz' saved [12715996/12715996]

root@shweta-VirtualBox:/tmp# cd ..
root@shweta-VirtualBox:/# cd opt
root@shweta-VirtualBox:/opt# cd tmp
-bash: cd: tmp: No such file or directory
root@shweta-VirtualBox:/opt# ls
google
root@shweta-VirtualBox:/opt# mkdir tomcat
root@shweta-VirtualBox:/opt# cd ..
root@shweta-VirtualBox:/# cd tmp
root@shweta-VirtualBox:/tmp# sudo tar -xvzf apache-tomcat-9.0.95.tar.gz -C /opt/tomcat --strip-components=1

```

Step 2: Download Apache Tomcat

- **cd /tmp** → Changes the current directory to the system's temporary folder.
- **wget** <https://archive.apache.org/dist/tomcat/tomcat-9/v9.0.95/bin/apache-tomcat-9.0.95.tar.gz> → Downloads Apache Tomcat 9.0.95 compressed file from the official Apache archive to the system.

Step 3: Create Directory for Tomcat Installation

- **cd ..** → Moves one level up from the current directory.
- **cd /opt** → Navigates to the /opt directory used for installing optional software.
- **mkdir tomcat** → Creates a new directory named tomcat to store the Tomcat files.
- **ls** → Lists directory contents to verify that the tomcat folder is created successfully.

Step 4: Extract Apache Tomcat Files

- **sudo tar -xvzf apache-tomcat-9.0.95.tar.gz -C /opt/tomcat --strip-components=1** → Extracts the downloaded Tomcat archive into /opt/tomcat directory by removing the top-level folder and placing all files directly inside it.

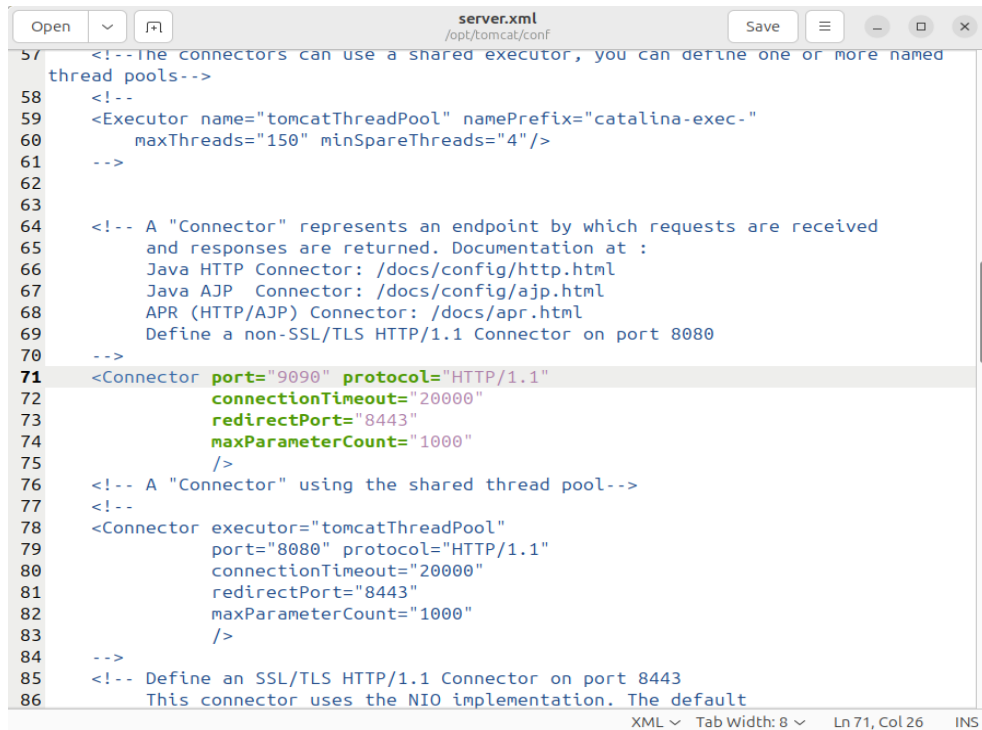
```

root@shweta-VirtualBox: /tmp
apache-tomcat-9.0.95/webapps/examples/websocket/index.xhtml
apache-tomcat-9.0.95/webapps/examples/websocket/snake.xhtml
apache-tomcat-9.0.95/webapps/host-manager/META-INF/context.xml
apache-tomcat-9.0.95/webapps/host-manager/WEB-INF/jsp/401.jsp
apache-tomcat-9.0.95/webapps/host-manager/WEB-INF/jsp/403.jsp
apache-tomcat-9.0.95/webapps/host-manager/WEB-INF/jsp/404.jsp
apache-tomcat-9.0.95/webapps/host-manager/WEB-INF/manager.xml
apache-tomcat-9.0.95/webapps/host-manager/WEB-INF/web.xml
apache-tomcat-9.0.95/webapps/host-manager/css/manager.css
apache-tomcat-9.0.95/webapps/host-manager/images/asf-Logo.svg
apache-tomcat-9.0.95/webapps/host-manager/images/tomcat.svg
apache-tomcat-9.0.95/webapps/host-manager/index.jsp
apache-tomcat-9.0.95/webapps/manager/META-INF/context.xml
apache-tomcat-9.0.95/webapps/manager/WEB-INF/jsp/401.jsp
apache-tomcat-9.0.95/webapps/manager/WEB-INF/jsp/403.jsp
apache-tomcat-9.0.95/webapps/manager/WEB-INF/jsp/404.jsp
apache-tomcat-9.0.95/webapps/manager/WEB-INF/jsp/connectorCerts.jsp
apache-tomcat-9.0.95/webapps/manager/WEB-INF/jsp/connectorCiphers.jsp
apache-tomcat-9.0.95/webapps/manager/WEB-INF/jsp/connectorTrustedCerts.jsp
apache-tomcat-9.0.95/webapps/manager/WEB-INF/jsp/sessionDetail.jsp
apache-tomcat-9.0.95/webapps/manager/WEB-INF/jsp/sessionsList.jsp
apache-tomcat-9.0.95/webapps/manager/WEB-INF/web.xml
apache-tomcat-9.0.95/webapps/manager/css/manager.css
apache-tomcat-9.0.95/webapps/manager/images/asf-Logo.svg
apache-tomcat-9.0.95/webapps/manager/images/tomcat.svg
apache-tomcat-9.0.95/webapps/manager/index.jsp
apache-tomcat-9.0.95/webapps/manager/status.xsd
apache-tomcat-9.0.95/webapps/manager/xform.xml
apache-tomcat-9.0.95/bin/catalina.sh
apache-tomcat-9.0.95/bin/ciphers.sh
apache-tomcat-9.0.95/bin/configtest.sh
apache-tomcat-9.0.95/bin/daemon.sh
apache-tomcat-9.0.95/bin/digest.sh
apache-tomcat-9.0.95/bin/makebase.sh
apache-tomcat-9.0.95/bin/setclasspath.sh
apache-tomcat-9.0.95/bin/shutdown.sh
apache-tomcat-9.0.95/bin/startup.sh
apache-tomcat-9.0.95/bin/tool-wrapper.sh
apache-tomcat-9.0.95/bin/version.sh

```

Step 5: Configure Tomcat Server

- **cd /opt/tomcat/conf** → Navigates to the Tomcat configuration directory.
- **ls** → Lists configuration files inside the directory.
- **gedit server.xml** → Opens the main Tomcat configuration file for editing.
- **Port Change (8080 → 9090)** → Modifies the default Tomcat port to 9090 to avoid conflicts with other services.



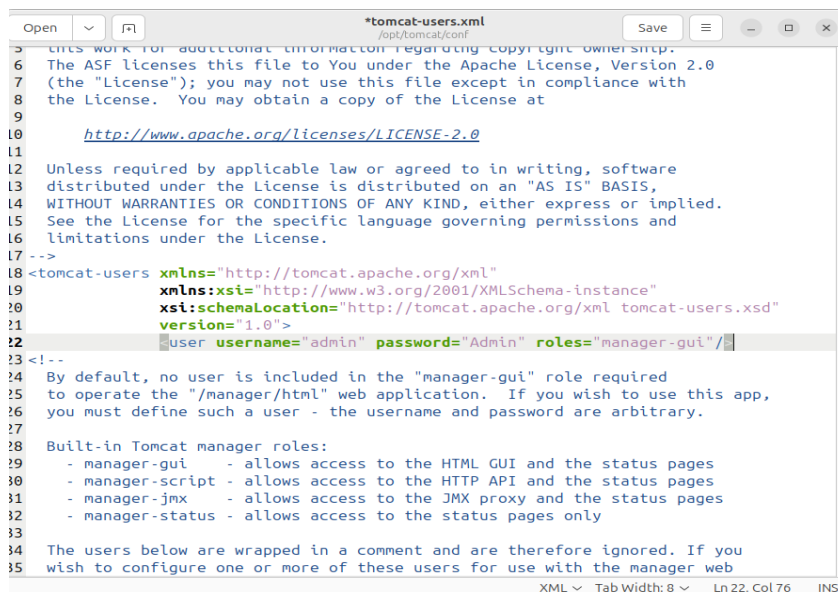
```

57 <!-- The connectors can use a shared executor, you can define one or more named
58 thread pools-->
59 <!--
60 <Executor name="tomcatThreadPool" namePrefix="catalina-exec-"
61     maxThreads="150" minSpareThreads="4"/>
62 -->
63
64 <!-- A "Connector" represents an endpoint by which requests are received
65 and responses are returned. Documentation at :
66 Java HTTP Connector: /docs/config/http.html
67 Java AJP Connector: /docs/config/ajp.html
68 APR (HTTP/AJP) Connector: /docs/apr.html
69 Define a non-SSL/TLS HTTP/1.1 Connector on port 8080
70 -->
71 <Connector port="9090" protocol="HTTP/1.1"
72     connectionTimeout="20000"
73     redirectPort="8443"
74     maxParameterCount="1000"
75 />
76 <!-- A "Connector" using the shared thread pool-->
77 <!--
78 <Connector executor="tomcatThreadPool"
79     port="8080" protocol="HTTP/1.1"
80     connectionTimeout="20000"
81     redirectPort="8443"
82     maxParameterCount="1000"
83 />
84 -->
85 <!-- Define an SSL/TLS HTTP/1.1 Connector on port 8443
86     This connector uses the NIO implementation. The default

```

Step 6: Configure Tomcat User Access

- **gedit tomcat-users.xml** → Opens the Tomcat user configuration file.
- **Add user with role** → Creates a user (admin) with password and assigns manager-gui role to access the Tomcat Manager dashboard.



```

5 This work for additional information regarding copyright ownership.
6 The ASF licenses this file to You under the Apache License, Version 2.0
7 (the "License"); you may not use this file except in compliance with
8 the License. You may obtain a copy of the License at
9
10 http://www.apache.org/licenses/LICENSE-2.0
11
12 Unless required by applicable law or agreed to in writing, software
13 distributed under the License is distributed on an "AS IS" BASIS,
14 WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
15 See the License for the specific language governing permissions and
16 limitations under the License.
17 -->
18 <tomcat-users xmlns="http://tomcat.apache.org/xml"
19     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
20     xsi:schemaLocation="http://tomcat.apache.org/xml tomcat-users.xsd"
21     version="1.0">
22     <user username="admin" password="Admin" roles="manager-gui"/>
23 <!--
24 By default, no user is included in the "manager-gui" role required
25 to operate the "/manager/html" web application. If you wish to use this app,
26 you must define such a user - the username and password are arbitrary.
27
28 Built-in Tomcat manager roles:
29 - manager-gui - allows access to the HTML GUI and the status pages
30 - manager-script - allows access to the HTTP API and the status pages
31 - manager-jmx - allows access to the JMX proxy and the status pages
32 - manager-status - allows access to the status pages only
33
34 The users below are wrapped in a comment and are therefore ignored. If you
35 wish to configure one or more of these users for use with the manager web

```

Step 7: Start Tomcat Server

- **cd /opt/tomcat/bin** → Navigates to the Tomcat binary directory.
- **ls** → Lists all available startup and shutdown scripts.
- **sh startup.sh** → Starts the Apache Tomcat server successfully.

```

uch file or directory)
root@shweta-VirtualBox:/opt/tomcat/conf# cd ..
root@shweta-VirtualBox:/opt/tomcat# cd bin
root@shweta-VirtualBox:/opt/tomcat/bin# ls
bootstrap.jar      ciphers.sh          daemon.sh           setclasspath.bat  startup.sh          version.bat
catalina.bat       commons-daemon.jar  digest.bat         setclasspath.sh   tomcat-juli.jar    version.sh
catalina.sh        commons-daemon-native.tar.gz digest.sh           shutdown.bat      tomcat-native.tar.gz
catalina-tasks.xml configtest.bat      makebase.bat       shutdown.sh        tool-wrapper.bat
ciphers.bat        configtest.sh       makebase.sh        startup.bat        tool-wrapper.sh
root@shweta-VirtualBox:/opt/tomcat/bin# sh startup.sh
Using CATALINA_BASE:   /opt/tomcat
Using CATALINA_HOME:   /opt/tomcat
Using CATALINA_TMPDIR: /opt/tomcat/temp
Using JRE_HOME:        /usr
Using CLASSPATH:       /opt/tomcat/bin/bootstrap.jar:/opt/tomcat/bin/tomcat-juli.jar
Using CATALINA_OPTS:
Tomcat started.
root@shweta-VirtualBox:/opt/tomcat/bin#

```

Step 8: Verify Tomcat Installation

- **Open Browser (<http://localhost:9090>)** → Confirms Tomcat server is running successfully by displaying the default Tomcat page.
- **Access Manager (<http://localhost:9090/manager/html>)** → Logs into the Tomcat Manager dashboard to manage and deploy web applications.

Home Documentation Configuration Examples Wiki Mailing Lists Find Help

Apache Tomcat/9.0.95

If you're seeing this, you've successfully installed Tomcat. Congratulations!

 Recommended Reading:
[Security Considerations](#) [How-To Manager Application](#) [How-To Clustering/Session Replication](#) [How-To](#)

[Server Status](#)
[Manager App](#)
[Host Manager](#)

Developer Quick Start
[Tomcat Setup](#) [First Web Application](#) [Realms & AAA](#) [JDBC DataSources](#) [Examples](#) [Servlet Specifications](#) [Tomcat Versions](#)

Managing Tomcat
 For security, access to the `manager` webapp is restricted. Users are defined in:
`$CATALINA_HOME/conf/tomcat-users.xml`
 In Tomcat 9.0 access to the manager application is split between different users.
[Read more...](#)

[Release Notes](#)
[Changelog](#)
[Migration Guide](#)
[Security Notices](#)

Documentation
[Tomcat 9.0 Documentation](#)
[Tomcat 9.0 Configuration](#)
[Tomcat Wiki](#)
 Find additional important configuration information in:
`$CATALINA_HOME/RUNNING.txt`
 Developers may be interested in:
[Tomcat 9.0 Bug Database](#)
[Tomcat 9.0 JavaDocs](#)
[Tomcat 9.0 Git Repository at GitHub](#)

Getting Help
FAQ and Mailing Lists
 The following mailing lists are available:
[tomcat-announce](#)
 Important announcements, releases, security vulnerability notifications. (Low volume).
[tomcat-users](#)
 User support and discussion.
[tomcat-dev](#)
 User support and discussion for [Apache Tomcat](#).
[tomcat-dev](#)
 Development mailing list, including commit messages.

[Other Downloads](#) [Tomcat Connectors](#) [Other Documentation](#) [Tomcat Connectors](#) [Get Involved](#) [Overview](#) [Miscellaneous](#) [Contact](#) [Apache Software Foundation](#)

Tomcat Web Application Manager

Message: OK

Manager
[List Applications](#) [HTML Manager Help](#) [Manager Help](#) [Server Status](#)

Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	None specified	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager	None specified	Tomcat Host Manager Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager	None specified	Tomcat Manager Application	true	1	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

Deploy
 Deploy directory or WAR file located on server

Context Path:

Version (for parallel deployment):

XML Configuration file path:

WAR or Directory with:

Jenkins Pipeline for MyMavenWebApp

Step 1: To deploy **MyMavenWebApp**, open the terminal, cd into the **MyMavenWebApp** folder, create a **Jenkinsfile** containing the pipeline code, and then commit and push it to GitHub.

pipeline {

agent any // Use any

available agent tools {

```
maven 'Maven' // Ensure this matches the name configured in Jenkins}
```

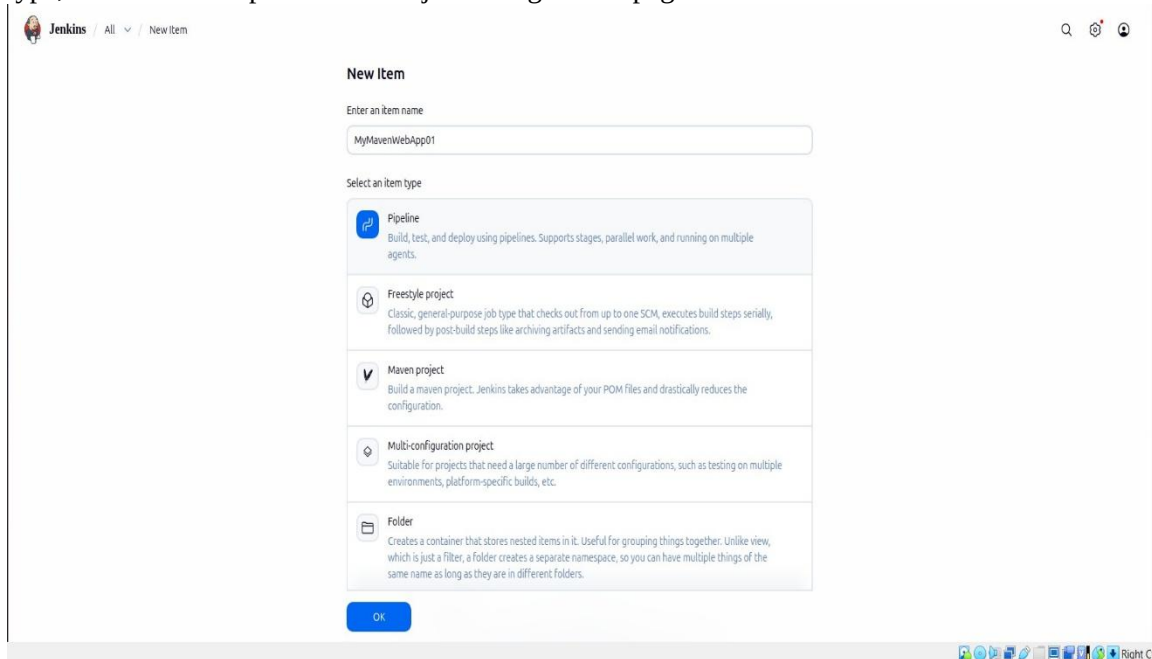
```
stages {  
  
  stage('Checkout') { steps {  
  
    git branch: 'master', url: 'https://github.com/numankhanssk/MymavenWebApp01.git'  
  
  }}  
  
  stage('Build') { steps {  
  
    sh 'mvn clean package' // Run Maven build  
  
  }  
}  
  
  stage('Test') { steps {  
  
    sh 'mvn test' // Run unit tests  
  
  }  
}  
  
  stage('Deploy WAR') { steps {  
  
    sh 'cp target/MymavenWebApp01.war /opt/tomcat/webapps/'  
  }  
}  
  
  post {  
  
    success {
```

```
echo 'Build and deployment successful!'
}
```

```
failure {
```

```
echo 'Build failed!'
}
}
}
```

Step 2: To deploy **MyMavenWebApp**, you first need to create a Jenkins pipeline job that will use the Jenkinsfile from your project repository. Then, go to the Jenkins home page and click on “**New Item**” to create a job, enter the item name (e.g., *MyMavenWebApp*), and select **Pipeline** as the project type. This option allows you to build, test, and deploy applications using a Jenkinsfile. After selecting the pipeline type, click “**OK**” to proceed to the job configuration page.



Step 3: In **MyMavenWebApp**, go to the **Pipeline** section and under **Definition**, select **Pipeline script from SCM**. Choose **Git** as the SCM, and under **Repository URL**, paste the GitHub URL of **MyMavenWebApp**. In the **Credentials** field, select the previously created credential (e.g., *GitHub Credential*). Finally, in **Script Path**, enter the name of the Jenkinsfile you created in the **MyMavenWebApp** folder and pushed to GitHub. Then click **Apply** and **Save** to

finalize the configuration. After clicking **Apply** and **Save**, go back to **MyMavenWebApp** in Jenkins and click **Build Now**; the build completes successfully.

The screenshot shows the Jenkins web interface for a job named 'MymavenWebApp'. The left sidebar contains navigation links: Status, Changes, Build Now, Configure, Delete Pipeline, Full Stage View, Stages, Rename, and Pipeline Syntax. The main area displays the 'Stage View' for the selected job. It includes a table of stage times and a 'Builds' section.

	Declarative: Checkout SCM	Declarative: Tool Install	Checkout	Build	Test	Deploy WAR	Declarative: Post Actions
Average stage times: (full run time: ~1min 35s)	4s	708ms	2s	54s	8s	1s	392ms
Build #1 Apr 15 09:11	4s	708ms	2s	54s	8s	1s	392ms

The 'Builds' section shows a single build with a green status icon and the text '#1 09:11'.